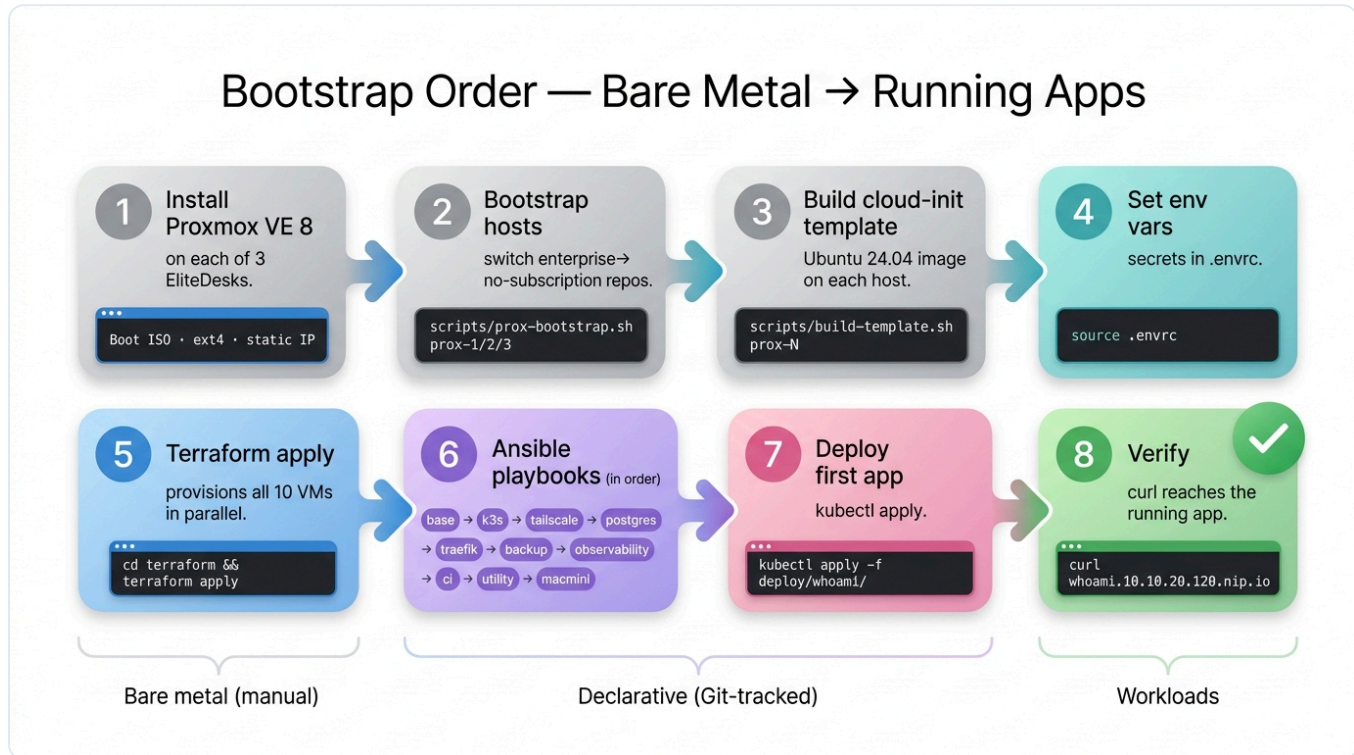


Homelab Runbook

Bootstrap, operate, and maintain · one repo, a handful of commands

Practical "if I had to rebuild this tomorrow" guide. Architecture lives in a separate doc.



8 numbered steps. Steps 1–3 are bare-metal-manual. 4–8 are Git-tracked.

1. Prerequisites

Hardware

- 3× HP EliteDesk mini PCs (≥16 GB RAM each, ≥256 GB SSD)
- 1× Mac Mini (Apple Silicon)
- 1× UniFi Cloud Gateway Max + UniFi Managed Switch + Nighthawk mesh (AP mode)
- 1× USB stick for the Proxmox installer

Accounts / external services

- **GitHub** — the repo (codephilip/homelab) + a PAT for the runner registration
- **Tailscale** — free tier is enough; one reusable pre-auth key
- **Let's Encrypt** — just an email address
- **Off-site backup** (optional) — Backblaze B2, S3, or GDrive for restic off-site

Local tooling on your Mac

```
brew install hashicorp/tap/terraform ansible helm kubectl direnv
```

Secrets — put these in `~/homelab/.envrc`

Variable	How to get it
<code>TF_VAR_proxmox_api_token_prox1/2/3</code>	On each Proxmox host: <code>pveum user token add terraform@pve provisioner --privsep 0</code>
<code>TF_VAR_ssh_public_key</code>	<code>cat ~/.ssh/id_ed25519.pub</code>
<code>TAILSCALE_AUTHKEY</code>	tailscale.com/admin/settings/keys → reusable + pre-approved
<code>POSTGRES_APP_PASSWORD</code>	<code>openssl rand -base64 24</code>
<code>RESTIC_PASSWORD</code>	<code>openssl rand -base64 32</code> · save it — required to restore
<code>GRAFANA_ADMIN_PASSWORD</code>	<code>openssl rand -base64 18</code>
<code>LETSENCRYPT_EMAIL</code>	Your real email address
<code>GITHUB_TOKEN</code> · <code>GITHUB_REPO</code>	GitHub PAT with Admin:write on the repo · <code>owner/repo</code>

Template lives at `.envrc.example`. Copy → `.envrc`, fill in, `source ~/.envrc` (or use `direnv`).

2. Full rebuild from scratch

Run the commands top-to-bottom. Each step is idempotent — safe to re-run if interrupted.

Step 1: Install Proxmox VE on each EliteDesk

Flash `proxmox-ve_8.x.iso` to a USB stick, boot each EliteDesk, ext4 filesystem, hostnames `prox-1/2/3`, static IPs `10.10.20.101/102/103`, gateway `10.10.20.1`.

Step 2: Bootstrap the Proxmox hosts (switch repos, apt update)

```
# On your Mac, after SSH-key copying to root@prox-1/2/3:
~/homelab/scripts/prox-bootstrap.sh prox-1
~/homelab/scripts/prox-bootstrap.sh prox-2
~/homelab/scripts/prox-bootstrap.sh prox-3
```

Step 3: Build the Ubuntu cloud-init template on each host

```
~/homelab/scripts/build-template.sh prox-1
~/homelab/scripts/build-template.sh prox-2
~/homelab/scripts/build-template.sh prox-3
```

Step 4: Set env vars

```
cd ~/homelab
cp .envrc.example .envrc
# edit .envrc, fill in all secrets
source .envrc
```

Step 5: Provision the 10 VMs with Terraform

```
cd ~/homelab/terraform
terraform init
terraform apply -auto-approve
terraform output ssh_targets    # confirms all 10 VMs
```

Step 6: Configure everything with Ansible (in this order)

```
cd ~/homelab/ansible
ansible-galaxy collection install -r requirements.yml
ansible all -m ping                                # all 10 VMs respond

ansible-playbook base.yml                          # baseline on every Linux VM
ansible-playbook k3s.yml                           # 1 cp + 2 workers, fetches kubeconfig
ansible-playbook tailscale.yml                     # subnet router
ansible-playbook postgres.yml                      # db1 + k8s ExternalName + Secret
ansible-playbook traefik.yml                       # ingress + cert-manager + LE issuers
ansible-playbook backup.yml                        # restic + cron + first snapshot
ansible-playbook observability.yml                 # Grafana + Prometheus + Loki + node-
exporters
ansible-playbook ci.yml                           # GitHub Actions runner
ansible-playbook utility.yml                       # AdGuard Home (browser wizard after)
ansible-playbook macmini.yml                       # Ollama + Python env
```

Step 7: Deploy the test app

```
export KUBECONFIG=~/.kube/home.yml
kubectl apply -f ~/homelab/deploy/whoami/
kubectl -n prod get pods -w
```

Step 8: Verify

```
curl http://whoami.10.10.20.120.nip.io
# pod hostname returned → end-to-end working
```

Manual finishing touches (one-time, browser/GUI)

- **Tailscale admin console:** approve `ts-router`'s 10.10.20.0/24 route, disable key expiry
- **Tailscale Mac app** on your laptop + Mac Mini: sign in
- **AdGuard wizard:** `http://10.10.20.132:3000` → admin port 8080 → set password
- **UniFi** → SERVERS DHCP → DNS Server: `10.10.20.132`
- **Off-site backup:** `ssh backup1; rclone config`

3. Common maintenance tasks

Add a new VM

1. Edit `terraform/locals.tf` → add an entry to `local.vms` with host/IP/cpu/ram/disk
2. (If new role) Add a group to `ansible/inventory.yml`
3. `cd terraform && terraform apply` → VM appears
4. Run whatever Ansible playbook is appropriate

Add a new app to k3s

1. Create `deploy/<app>/manifest.yaml` with Deployment + Service + Ingress
2. For DB access, mount the existing `postgres-app` Secret (see `deploy/README.md`)
3. `kubectl apply -f deploy/<app>/`
4. Reach it at `http://<app>.<worker-ip>.nip.io` or your real domain

Update k3s version

Edit `k3s_version` in `ansible/k3s.yml` → run `ansible-playbook k3s.yml`. The install script no-ops if version matches; if not, it upgrades.

Resize a VM

Edit `cores` / `memory` / `disk` in `terraform/locals.tf` → `terraform apply`. Some changes (memory) are hot-applied; others (CPU, disk) need a VM reboot.

Restore Postgres from backup

```
ssh backup1
. /etc/backup.env
restic snapshots                # list snapshots
restic restore latest --target /tmp/restore # pull the latest
gunzip -c /tmp/restore/tmp/pg-*.sql.gz | \
    psql -h 10.10.20.111 -U appuser app      # restore
```

Rotate a secret

Generate new value, replace in `.envrc`, `source ./envrc`, re-run the relevant playbook (e.g. `postgres.yml` for DB password). For Proxmox tokens, regenerate on the host first.

4. Repo structure — what owns what

Path	Owns	Touch when...
<code>scripts/prox-bootstrap.sh</code>	Proxmox host repo config + apt	New host, or stale apt
<code>scripts/build-template.sh</code>	Ubuntu cloud-init template (VM 9000)	New host, or new Ubuntu LTS
<code>terraform/locals.tf</code>	Single source of truth for IP / sizing plan	Add / resize / move VMs
<code>terraform/prox-N.tf</code>	VM lifecycle on each host	Rare — uses <code>for_each</code> over locals
<code>ansible/inventory.yml</code>	Host list + groups for Ansible	New host or new role group
<code>ansible/base.yml</code>	Common config for every Linux VM	Add shared tools, change TZ, etc.
<code>ansible/k3s.yml</code>	k3s install / upgrade	Version bumps
<code>ansible/postgres.yml</code>	Postgres + k8s Service + Secret	DB password rotation, version bump
<code>ansible/traefik.yml</code>	Ingress + cert-manager + LE issuers	Helm value tweaks, new ClusterIssuer
<code>ansible/backup.yml</code>	restic repo, backup script, cron	Retention policy changes
<code>ansible/observability.yml</code>	Grafana + Prom + Loki + node-exporters	New scrape targets, dashboards
<code>ansible/ci.yml</code>	GitHub Actions self-hosted runner	Switch repos / org-level
<code>ansible/utility.yml</code>	AdGuard Home	Adding internal hostnames
<code>ansible/macmini.yml</code>	Mac Mini brew packages + Ollama	New AI tools
<code>deploy/<app>/</code>	Per-app k8s manifests	New deployments
<code>.envrc</code> (gitignored)	All secrets, never committed	Token / password rotation
<code>.kube/home.yml</code> (gitignored)	kubectl credentials	Re-fetched by <code>k3s.yml</code> when run

The rule: if it's reproducible, it lives in this repo. If it's a secret or a generated artifact, it lives in `.envrc` or under a gitignored path. There's no Step 4½ on a sticky note somewhere.